

THE AI-NATIVE STARTUP HANDBOOK

How Founders Must Rethink Everything in an
AI-First World

By Nitin Gupta

Founder & CEO, FlytBase

July 2025

Living Products vs. Dead Products—The Creeper Model

I was walking through my neighborhood in Pune last weekend when I noticed something that perfectly captured what I've been trying to articulate about the future of software.

There were two houses side by side. The first house had a beautiful artificial plant display in the front yard—perfectly manicured, symmetrical, and unchanging. It looked exactly the same as it did six months ago when the owners first installed it.

The second house had a creeper vine growing along its wall. Six months ago, it was just a small plant. Now it had spread across the entire facade, adapting to the contours of the building, finding its way around windows and architectural features, creating a unique pattern that could never be replicated.

Both were "green." Both served a decorative purpose. But only one was alive.

As I stood there, I realized I was looking at the perfect metaphor for the two types of software that will exist in the intelligence explosion era: dead products and living products.

The Artificial Plant Problem

Most software today is like that artificial plant—static, unchanging, requiring manual intervention to improve or adapt.

You build a feature, deploy it, and it does exactly what you programmed it to do. If you want it to work better, you have to manually update the code. If you want it to handle new scenarios, you have to manually add new logic. If you want it to adapt to different users, you have to manually create different configurations.

This approach made sense when software was expensive to build and modify. You created something that worked for the majority of use cases, and users adapted their workflows to fit your software's limitations.

But in the intelligence explosion, this static approach becomes a liability.

The Product Evolution Factor (PEF)

To understand the difference between dead and living products, I've started thinking in terms of what I call the Product Evolution Factor (PEF).

PEF $\leq$ 1: Dead Products These are products that don't evolve without manual effort. They might even degrade over time (PEF $<$ 1) as the environment around them changes but they remain static.

Think about traditional software applications. They do exactly what they were programmed to do, nothing more. Over time, as user needs evolve and the competitive landscape changes, these products become less useful unless someone manually updates them.

PEF > 1: Living Products These are products that get better and more useful over time without manual intervention. They learn from usage patterns, adapt to user preferences, and evolve their capabilities based on the environment they operate in.

This isn't about having a learning algorithm somewhere in your codebase. It's about building products that fundamentally improve their value proposition over time through interaction with users and data.

The Creeper Model

Living products behave like creepers—they adapt to their environment and grow in ways that create unique value for each context they're planted in.

Just as the same creeper seed will grow differently depending on the wall it's planted against, the same software should evolve differently based on the organization or user it serves.

At FlytBase, we've started building our drone platform with this creeper model in mind. Instead of creating a one-size-fits-all

solution, we're building a system that learns from each deployment and becomes more effective for that specific use case over time.

A FlytBase system deployed for security surveillance learns different patterns than one deployed for agricultural monitoring. The same core platform, but it grows and adapts to become uniquely valuable for each application.

The Learning Loop Architecture

The key to building living products is creating what I call "learning loops"—systems that continuously improve based on real-world usage.

Traditional Architecture: Input → Processing → Output

Living Product Architecture: Input → Processing → Output → Learning → Improved Processing

The learning component analyzes outcomes, identifies patterns, and feeds insights back into the processing layer. Over time, the product becomes more effective at achieving the desired outcomes.

But here's the crucial part: this learning happens automatically, without human intervention. The product evolves on its own.

The Customization Without Configuration Problem

One of the most powerful aspects of the creeper model is that it enables mass customization without manual configuration.

Traditional software requires you to manually configure it for different use cases. You create settings, options, and parameters that users can adjust to fit their needs.

Living products customize themselves. They observe how they're being used, what outcomes are desired, and what environmental factors matter, then adapt their behavior accordingly.

At FlytBase, instead of asking customers to configure dozens of parameters for their specific use case, our system observes their operations and automatically optimizes for their particular environment and objectives.

The result is software that becomes uniquely valuable to each customer without requiring manual customization effort.

The Network Effect of Learning

But here's where living products become truly powerful: when learning loops are connected across multiple deployments, the entire network gets smarter.

Insights learned from one deployment can inform improvements across all deployments, while still maintaining the unique adaptations that make each deployment valuable in its specific context.

This creates a compound learning effect where the value of the product increases exponentially with the number of users, not just linearly.

The Data Advantage

Living products also create a different relationship with data.

Dead products consume data to produce outputs. Living products consume data to improve their ability to produce better outputs over time.

This means that living products become more valuable the longer they operate, while dead products maintain constant value (or even decrease in value as they become outdated).

For customers, this creates a switching cost that's based on value rather than inconvenience. They don't stay because it's hard to leave—they stay because the product keeps getting better for their specific needs.

The Competitive Implications

The distinction between dead and living products creates a new competitive dynamic.

Companies building dead products compete on features and functionality. They try to build the best possible product at launch and then incrementally improve it through manual updates.

Companies building living products compete on learning velocity and adaptation capability. They try to build products that get better faster than their competitors' products.

This creates a divergence where the two approaches become increasingly incomparable over time. It's not just that living products are better—they're playing a different game entirely.

The FlytBase Evolution

Let me give you a concrete example of how this plays out at FlytBase.

Old (Dead Product) Approach: We built drone software with predefined flight patterns, obstacle avoidance algorithms, and mission parameters. Customers would configure these settings for their specific use case, and the software would execute exactly as programmed.

New (Living Product) Approach: Our AI system observes each mission, learns from successes and failures, and continuously optimizes its approach. A system deployed for warehouse inventory learns different patterns than one deployed for search and rescue operations.

The new system doesn't just perform better—it becomes uniquely valuable to each customer in ways that would be impossible to replicate manually.

The Development Paradigm Shift

Building living products requires a fundamental shift in how we think about software development.

Traditional Development: Build → Test → Deploy → Maintain

Living Product Development: Build → Deploy → Learn → Evolve → Learn → Evolve...

The development process never really ends. Instead of shipping a finished product, you ship a learning system that continuously improves itself.

This requires different skills, different architectures, and different organizational structures. You need teams that can build systems that learn, not just systems that execute.

The User Experience Revolution

Living products also create a fundamentally different user experience.

With dead products, users have to learn how the software works and adapt their behavior to fit the software's limitations.

With living products, the software learns how users work and adapts its behavior to fit their preferences and patterns.

This creates a user experience that gets better over time, rather than one that users simply get used to over time.

The Measurement Challenge

But living products also create new challenges in how we measure success.

Traditional software metrics focus on usage, performance, and user satisfaction at specific points in time.

Living products require metrics that capture improvement over time, adaptation effectiveness, and the compound value created through learning.

At FlytBase, we've had to develop entirely new ways of measuring our product's success that account for its evolutionary nature.

The Organizational Implications

Building living products also requires different organizational structures.

Traditional product teams are organized around building and maintaining features.

Living product teams need to be organized around understanding user outcomes, designing learning systems, and continuously improving the product's ability to achieve those outcomes.

This requires closer collaboration between product, engineering, and data teams, and a different approach to product management that focuses on learning velocity rather than feature delivery.

The Ethical Considerations

Living products also raise new ethical considerations.

When software adapts and learns from user behavior, questions arise about transparency, control, and unintended consequences.

Users need to understand how the product is learning and evolving. They need some level of control over that evolution. And companies need to ensure that the learning process doesn't create harmful or biased outcomes.

At FlytBase, we've had to develop new frameworks for ensuring that our learning systems evolve in ways that are beneficial and aligned with user intentions.

The Future of Software

The distinction between dead and living products isn't just a technical choice—it's a fundamental shift in what software can be.

Dead products are tools that do what you tell them to do.

Living products are partners that learn to help you achieve your goals more effectively over time.

This shift changes everything about how we design, build, market, and support software products.

The Transition Strategy

So how do you transition from building dead products to living products?

Start with Learning Loops: Identify one area where your product could learn from user behavior and improve over time. Build a simple learning loop around that area.

Measure Evolution: Develop metrics that capture how your product improves over time, not just how it performs at any given moment.

Design for Adaptation: Instead of building fixed solutions, build systems that can adapt their behavior based on observed outcomes.

Embrace Uncertainty: Accept that you can't predict exactly how your product will evolve. Build systems that can learn and adapt in ways you didn't anticipate.

Focus on Outcomes: Instead of optimizing for feature completeness, optimize for the product's ability to achieve user outcomes more effectively over time.

The Compound Advantage

But here's the most important insight: living products create compound advantages that become very difficult for competitors to overcome.

The longer a living product operates, the more valuable it becomes to its users. The more users it has, the faster it learns and improves.

This creates a flywheel effect where success compounds over time, rather than just accumulating linearly.

The Choice

Every software company faces this choice: build dead products or living products.

You can create software that does what you program it to do, or you can create software that learns to do what users need it to do.

You can compete on features and functionality, or you can compete on learning velocity and adaptation capability.

You can build tools, or you can build partners.

The intelligence explosion makes living products not just possible, but inevitable. The companies that understand this first will have advantages that compound over time.

The question isn't whether your product should be alive or dead. The question is how quickly you can bring it to life.

Are you building artificial plants or creepers?

The future belongs to the gardeners who understand the difference.